DOSSIER DE ESTUDIO / PRÁCTICAS DE SEGURIDAD WEB

WEBGOAT GUÍA COMPLETA PARA EL EXAMEN

Teoría clara + procedimiento de cada ejercicio + Burp Suite + pruebas de comprobación

Preparado a partir de: «WebGoat Exercicis Victor Risco.pdf», 98 páginas, con tus procedimientos, capturas y anotaciones.

Enfoque: entender qué falló, cómo lo comprobaste, qué señal encontraste y cómo repetir el razonamiento en un examen.

Importante: las rutas, flags y valores de ejemplo pertenecen a prácticas del laboratorio. Pueden cambiar en una instalación o intento distinto.

Cómo usar esta guía

Primera vuelta: lee las explicaciones. Segunda vuelta: repite cada práctica sin mirar el resultado. Última vuelta: utiliza la hoja de repaso y el simulacro final.

Edición de estudio · Octubre de 2026

Cómo está organizada la guía

He seguido los bloques del índice de tu PDF y los números de ejercicio que anotaste. En cada práctica encontrarás el objetivo, los pasos documentados, una explicación del mecanismo y una forma de comprobar el resultado.

> **Diferencia entre fuente y ampliación**
>
> «En tu práctica» significa que el procedimiento está recogido en tu documento original. «Aclaración técnica» añade contexto didáctico para entenderlo; no pretende sustituir lo que demuestra tu captura.

Temario y prioridad

Bloque	PDF original	En esta guía
HTTP + Burp + Developer Tools	pp. 3-8	Herramientas y lectura HTTP
CIA Triad	pp. 8-10	Conceptos esenciales
Acceso roto (sesiones, IDOR, funciones)	pp. 11-19	Tres prácticas distintas
Cookies y Crypto Basics	pp. 20-26	Decodificación y autenticación
SQL Injection + Blind	pp. 32-56	Sintaxis, UNION e inferencia
XSS	pp. 57-65	Reflected, DOM, Stored
Path Traversal + Zip Slip	pp. 70-78	Rutas, encoding y ZIP
CSRF	pp. 79-85	Ejercicios 3, 4, 7 y 8
Security Misconfiguration	pp. 86-87	Errores y fugas

Marcado por ti como que no entra: Crypto Basics 6 y 8; SQL Injection (mitigation); XSS (mitigation); XXE. La guía incluye solo referencias breves a defensas cuando ayudan a entender el fallo. Esta selección procede de tus anotaciones, no de una confirmación del profesor.

Índice de capítulos

Cómo está organizada la guía 2

01 · Cómo analizar una petición HTTP 4

02 · Burp Suite paso a paso 5

03 · HTTP Basic y HTTP Proxies 6

04 · Developer Tools y CIA Triad 7

05 · Hijack a Session 8

06 · IDOR: acceso a objetos de otros usuarios 10

07 · Missing Function Level Access Control 11

08 · Spoofing an Authentication Cookie 13

09 · Crypto Basics: Base64, XOR y hashes 14

10 · SQL: fundamentos para los ejercicios intro 16

11 · SQL Injection avanzada: UNION SELECT 18

12 · Blind SQL Injection: inferir información 19

13 · XSS: Reflected, DOM y Stored 22

14 · Path Traversal: cómo funcionan las rutas 24

15 · Zip Slip: Path Traversal dentro de un ZIP 26

16 · CSRF: peticiones con la sesión de otro 27

17 · Security Misconfiguration 29

18 · Tabla de decisión y checklist de Burp 31

19 · Hoja de memoria: comandos, payloads y diferencias 32

20 · Simulacro de examen: 20 preguntas 33

21 · Respuestas del simulacro 33

Apéndice · Qué falta o no está confirmado en el original 34

01 · Cómo analizar una petición HTTP

En tu PDF: pp. 3-6

Anatomía de una petición

En WebGoat el navegador envía peticiones HTTP al servidor. Con Burp Suite puedes ver exactamente qué método, ruta, cabeceras, cookies y datos llegan. Así identificas qué parámetro controla el usuario.

```
POST /WebGoat/ruta-del-ejercicio HTTP/1.1
Host: 127.0.0.1:8081
Cookie: JSESSIONID=VALOR-DE-LA-SESION
Content-Type: application/x-www-form-urlencoded
nombre=Victor&valor=123
```

Parte	Qué significa
POST	Método de la petición. Otros comunes: GET, PUT, DELETE.
/WebGoat/...	Ruta solicitada en el servidor.
Cookie	Identificador u otros datos asociados a la sesión.
Content-Type	Describe el formato del cuerpo que se envía.
Línea en blanco	Separa cabeceras y cuerpo en HTTP/1.x.
Body	Contiene los datos enviados en este ejemplo.

GET frente a POST: el examen suele jugar con esto

En tu ejercicio de proxy se pasa un parámetro desde el body de POST a la URL de GET. El objetivo es demostrar que puedes modificar la petición real. GET suele usar el query string para parámetros; POST suele tener cuerpo, pero ninguno de los dos es automáticamente seguro ni privado.

```
GET /WebGoat/ruta-del-ejercicio?changeMe=123 HTTP/1.1
Host: 127.0.0.1:8081
x-request-intercepted: true
```

Regla útil: después de ? comienza la query, y & separa parámetros: `?id=3&name=Victor`. Los valores que aparecen en una URL pueden quedar en registros o historial.

02 · Burp Suite paso a paso

En tu PDF: pp. 3-5, 11-21, 54-55, 73-83

Los tres módulos que más utilizaste

Módulo	Para qué lo utilizaste
Proxy / HTTP history	Interceptar o localizar una petición enviada por WebGoat.
Repeater	Modificarla, enviarla varias veces y comparar respuestas.
Intruder	Probar rangos numéricos, caracteres o valores de una posición.

Flujo general que puedes repetir en casi todos los ejercicios

1. Abre el ejercicio y realiza una acción (enviar formulario, pulsar un botón o cargar un perfil).

2. Encuentra la petición en Proxy → HTTP history o detenla con Intercept. Comprueba método, ruta y parámetros.

3. Haz Send to Repeater para probar una modificación controlada. Envía con Send y compara la respuesta con la original.

4. Si necesitas muchas pruebas, manda la petición a Intruder. Selecciona únicamente la parte variable y configura un conjunto de valores.

5. Revisa Status code, Length, texto de la respuesta y cambios funcionales. Vuelve a verificar manualmente el candidato.

> **Clave de Burp Intruder**
>
> Una longitud diferente en la respuesta indica que algo cambió. No demuestra por sí sola que el ataque haya tenido éxito: puede ser un error, un mensaje distinto, un token variable o una respuesta real.

Cabeceras que pueden aparecer en los ejercicios

Cabecera	Interpretación
Referer	URL desde la que se originó la navegación o la petición, cuando el navegador la envía.
Origin	Origen de seguridad (esquema, host y puerto) declarado en determinados tipos de petición.
Accept	Formato de respuesta que el cliente prefiere recibir.
Content-Type	Formato del cuerpo enviado o recibido.
Cookie	Datos de estado enviados por el navegador al servidor.
x-request-intercepted	Cabecera adicional que WebGoat comprueba en el ejercicio HTTP Proxies 6.

03 · HTTP Basic y HTTP Proxies

En tu PDF: pp. 3-5

HTTP Basic, ejercicio 3 | PDF pp. 3

Objetivo: Construir/modificar una petición HTTP para ajustarla a las condiciones del enunciado.

Procedimiento de tu práctica

1. Captura la petición POST de la práctica.
2. Pásala a Repeater o edítala en Burp.
3. Ajusta los componentes exigidos por el enunciado: método, ruta, cabeceras y contenido.
4. Envía la petición y verifica la respuesta de WebGoat.

Por qué funciona: Burp permite enviar un HTTP distinto del generado inicialmente por la interfaz.

Cómo reconocer el resultado: WebGoat marca el ejercicio como completado si la petición satisface las condiciones.

Errores frecuentes: Tu archivo solo resume «cambiar la petición POST y poner lo que te dice»; los valores exactos se ven en la captura, no están transcritos en el texto. No memorices parámetros que no puedas leer.

HTTP Basic, ejercicio 4 | PDF pp. 4

Objetivo: Localizar el valor «magic» en la comunicación HTTP.

Procedimiento de tu práctica

1. Reproduce la acción de la página.
2. Busca la petición POST relacionada.
3. Examina los parámetros y la respuesta hasta identificar el número que WebGoat llama «magic».
4. Introduce el valor en la actividad.

Por qué funciona: El valor buscado viaja o aparece en la comunicación real, aunque la pantalla no lo destaque.

Cómo reconocer el resultado: El número hallado coincide con la comprobación del ejercicio.

HTTP Proxies, ejercicio 6 | PDF pp. 5

Objetivo: Convertir la petición POST original en GET y añadir una cabecera de verificación.

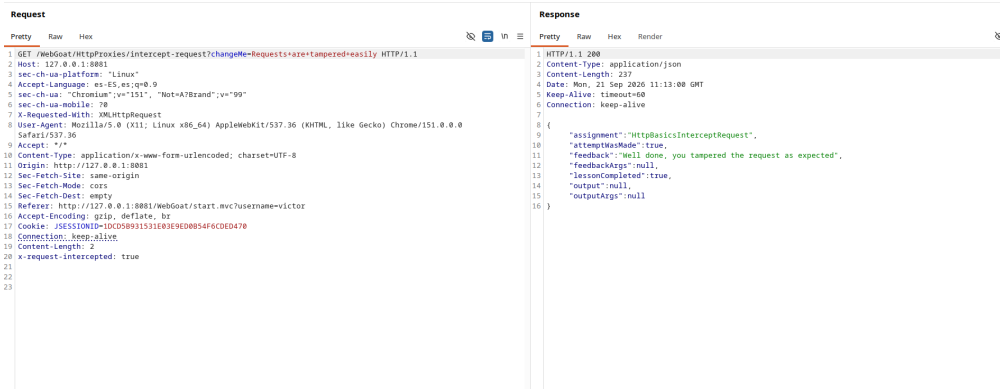
Procedimiento de tu práctica

1. Captura la petición POST.
2. Sustituye POST por GET y coloca el parámetro changeMe en la URL, tras ?.
3. Elimina ese dato del body si ya no debe enviarse allí.
4. Añade `x-request-intercepted: true`.
5. Envía y observa la comprobación de WebGoat.

Por qué funciona: Demuestras que la petición puede interceptarse y reescribirse antes de que llegue al servidor.

Cómo reconocer el resultado: La actividad acepta el cambio de método, el parámetro y la cabecera.

Errores frecuentes: No confundas la cabecera adicional con la línea en blanco. La primera es una comprobación del ejercicio; la segunda delimita el cuerpo en HTTP/1.x.



Captura del PDF original, pág. 5: Burp muestra la petición cambiada y la respuesta de la práctica HTTP Proxies 6.

04 · Developer Tools y CIA Triad

En tu PDF: pp. 6-10

Pestañas del navegador: para qué sirve cada una

Pestaña	Uso concreto
Elements	Ver HTML, atributos, clases, formularios y contenido modificado.
Console	Ejecutar una llamada JavaScript y leer logs, valores o errores.
Sources	Consultar ficheros JavaScript y descubrir cómo trata datos o rutas.
Network	Ver peticiones, cabeceras, Payload, Response y tiempos.

Developer Tools, ejercicio 4 | PDF pp. 6

Objetivo: Encontrar un número mediante una función JavaScript.

Procedimiento de tu práctica

1. Abre DevTools (F12).
2. Identifica la función que indica el ejercicio o que aparece en el código fuente.
3. Llama a esa función desde la consola.
4. Recupera el número que devuelve o muestra.

Por qué funciona: La consola permite interactuar con las funciones presentes en la página.

Cómo reconocer el resultado: La salida proporciona el valor pedido por WebGoat.

Errores frecuentes: No asumas que una función devolverá siempre un valor: algunas solo escriben en consola y devuelven undefined.

Developer Tools, ejercicio 6 | PDF pp. 8

Objetivo: Localizar un valor enviado en una petición del navegador.

Procedimiento de tu práctica

1. Abre Network.
2. Pulsa GO en la página de WebGoat.
3. Selecciona la solicitud llamada «Network» que documentaste.
4. Abre la pestaña Payload y localiza el número.

Por qué funciona: Network muestra los datos enviados sin depender de cómo se representen en el HTML.

Cómo reconocer el resultado: El valor del payload coincide con lo que pide el formulario.

CIA Triad: tres objetivos de seguridad

Propiedad	Definición y ejemplo de tu temario
Confidentiality	Solo los autorizados pueden consultar datos. Fallo: filtración de contraseñas.
Integrity	Los datos deben permanecer correctos y sin cambios no autorizados. Fallo: alterar registros.
Availability	El servicio debe poder utilizarse cuando se necesita. Fallo: caída por DoS.

Asocia cada propiedad con defensas: confidencialidad (control de acceso, cifrado), integridad (permisos, controles, hashes) y disponibilidad (redundancia, mantenimiento, protección frente a saturación).

05 · Hijack a Session

En tu PDF: pp. 11-13

Qué vulnerabilidad demuestra

El secuestro de sesión ocurre cuando alguien consigue utilizar un identificador de sesión válido de otra persona. En tu laboratorio se observa una cookie con valores susceptibles de búsqueda: una parte puede ser predicha/acotada comparando respuestas. La actividad es una demostración sobre el mecanismo de sesión de WebGoat.

Práctica: Repeater + Intruder | PDF pp. 11-13

Objetivo: Encontrar la cookie candidata que permite entrar en una sesión ajena del laboratorio.

Procedimiento de tu práctica

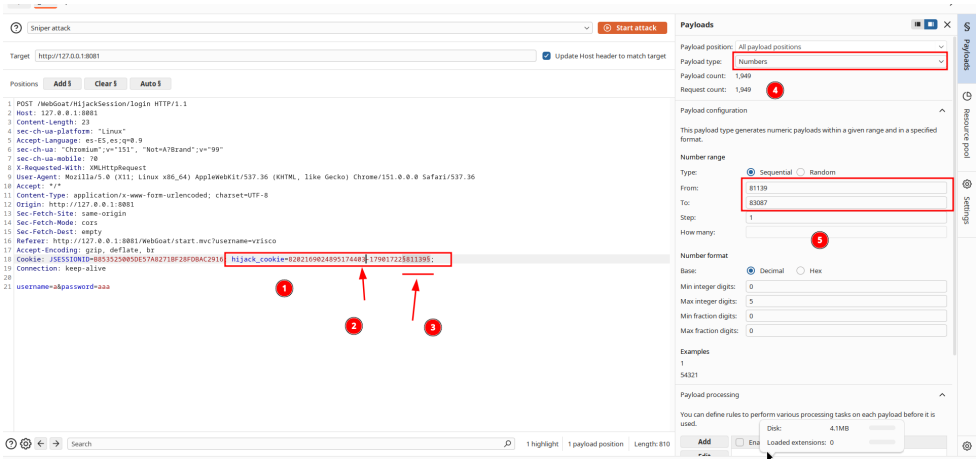
1. Abre el formulario del ejercicio y captura la petición correspondiente.
2. Envíala a Repeater y repite Send para observar qué componentes de la cookie cambian.
3. En tus notas el número que cambia alrededor del ejemplo es 4403, con valores vecinos 4402 y 4404.
4. Envía la petición a Intruder y selecciona únicamente los últimos cinco dígitos de la parte de cookie que se quiere probar.
5. Configura una carga numérica alrededor del intervalo inferido y ejecuta las solicitudes de prueba.

6. Ordena los resultados por Length, abre los que son distintos y confirma cuál responde como una sesión válida.

Por qué funciona: La actividad representa sesiones con un identificador cuya estructura facilita probar candidatos. Al variar la parte adecuada puedes observar diferencias en el servidor.

Cómo reconocer el resultado: La respuesta distinta debe mostrar el efecto funcional de estar en otra sesión: no basta ver una longitud anómala.

Errores frecuentes: No marques como payload toda la cookie si solo cambian cinco cifras: destruirías la estructura que observaste.



Captura del PDF original, pág. 12: Configuración de Intruder sobre el segmento variable; la numeración marcada pertenece a tu intento.

> **Pregunta probable**
>
> ¿Por qué usamos Intruder y no Repeater? Repeater sirve para comparar pruebas puntuales. Intruder recorre automáticamente un rango de valores del mismo parámetro.

Si el resultado no es el esperado

- Comprueba que la cookie esté en la cabecera Cookie de la solicitud correcta.
- Asegúrate de seleccionar solo el tramo variable con los marcadores de Intruder.
- Si varias respuestas difieren en Length, compara sus cuerpos: algunos cambios pueden deberse a avisos o tokens.

06 · IDOR: acceso a objetos de otros usuarios

En tu PDF: pp. 13-15

Concepto que hay que saber explicar

IDOR (Insecure Direct Object References) significa que una aplicación acepta un identificador de objeto (usuario, archivo, registro) sin comprobar suficientemente si el usuario autenticado está autorizado a consultarlo o modificarlo. Cambiar un ID no es en sí un ataque: el fallo aparece cuando el servidor entrega un recurso ajeno.

IDOR, ejercicio 3: observar diferencias | PDF pp. 13

Objetivo: Encontrar los identificadores o atributos que devuelve la aplicación.

Procedimiento de tu práctica

1. Pulsa el botón indicado en WebGoat.
2. En Network o Burp busca la petición relacionada.
3. Abre la respuesta y anota los campos de usuario, identificadores y rutas relevantes.

Por qué funciona: La interfaz muestra solo una parte; la respuesta HTTP puede contener referencias a objetos.

Cómo reconocer el resultado: Puedes identificar los valores necesarios para los ejercicios siguientes.

IDOR, ejercicio 4: Guessing & Predicting Patterns | PDF pp. 14

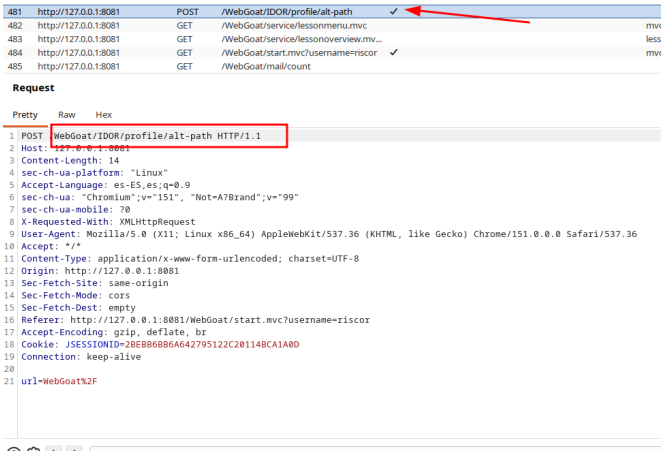
Objetivo: Probar una referencia directa perteneciente a «tomcat».

Procedimiento de tu práctica

1. Envía el formulario para provocar la petición.
2. Localiza la petición alt-path en el historial.
3. Mándala a Repeater.
4. Cambia en la ruta o referencia el identificador actual por el de tomcat obtenido de la respuesta.
5. Envía y analiza qué datos devuelve.

Por qué funciona: La aplicación reutiliza el ID indicado por el cliente para buscar otro registro; el laboratorio no lo protege correctamente.

Cómo reconocer el resultado: La respuesta contiene un registro distinto, asociado al identificador probado.



Captura del PDF original, pág. 14: Solicitud de «alt-path» en Burp para la prueba con un identificador distinto.

IDOR, ejercicio 5: Playing with the Patterns | PDF pp. 15

Objetivo: Descubrir otros valores de identificador siguiendo un patrón.

Procedimiento de tu práctica

1. Captura una petición con ID y envíala a Intruder.
2. Coloca marcadores alrededor de los dos últimos dígitos observados en tus notas.
3. Configura un rango Numbers hasta 99 según el reto.
4. Ejecuta, revisa respuestas y valida manualmente los objetos hallados.

Por qué funciona: Los identificadores pueden seguir patrones y ser enumerables. La vulnerabilidad real es la falta de comprobación de autorización.

Cómo reconocer el resultado: Una respuesta confirma que se han consultado datos que no deberían estar accesibles desde ese usuario.

> **No confundir**
>
> IDOR = acceder a otro objeto alterando su referencia. El siguiente bloque, Missing Function Level Access Control, trata de acceder a otra función (por ejemplo, una ruta administrativa).

07 · Missing Function Level Access Control

En tu PDF: pp. 16-19

Aquí la aplicación no protege bien ciertas funciones o endpoints. Aunque un botón no salga en el menú, la ruta puede seguir existiendo; ocultar la interfaz no equivale a comprobar permisos en el servidor.

Ejercicio 2: localizar funciones ocultas | PDF pp. 16

Objetivo: Encontrar rutas que la interfaz no ofrece de forma visible.

Procedimiento de tu práctica

1. Abre DevTools y examina el HTML y los componentes mostrados.
2. Despliega elementos donde veas enlaces, rutas o funciones definidas.
3. Anota las dos entradas que en tus capturas no aparecen en el menú normal.
4. Prueba esas rutas dentro del ejercicio.

Por qué funciona: El navegador recibe referencias que podrían ser accesibles incluso si la aplicación no ofrece un botón.

Cómo reconocer el resultado: WebGoat responde a las funciones no expuestas en la navegación normal.

Ejercicio 3: método y ruta protegida | PDF pp. 17

Objetivo: Consultar una función mediante una petición HTTP modificada.

Procedimiento de tu práctica

1. Captura la solicitud de la sección.
2. Envía a Repeater y cambia el POST original por GET si el endpoint que vas a consultar lo requiere.
3. Modifica la ruta según la pista del ejercicio.
4. Solicita una respuesta JSON mediante la cabecera adecuada y revisa el resultado.

Por qué funciona: Se invoca directamente un endpoint que debería restringirse, fuera del flujo normal de la interfaz.

Cómo reconocer el resultado: La respuesta devuelve información de la función buscada.

```
GET /WebGoat/access-control/users HTTP/1.1
Accept: application/json
```

Ejercicio 4: lista de usuarios y hash | PDF pp. 18-19

Objetivo: Consultar una lista de usuarios y observar el efecto de una segunda petición.

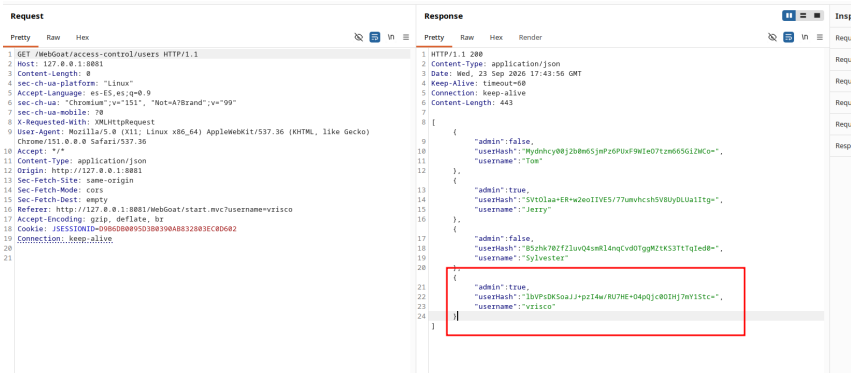
Procedimiento de tu práctica

1. Tus notas empiezan con la llamada GET al endpoint `/WebGoat/access-control/users`.
2. La respuesta JSON contiene registros y hashes; documentas copiar los campos necesarios.
3. Después describes una petición POST con JSON, cambiando el atributo `userHash` por `password`.
4. Vuelves a hacer GET y buscas el nuevo usuario registrado y su hash.

Por qué funciona: Se está manipulando una función expuesta indebidamente y observando cómo cambia su colección de usuarios.

Cómo reconocer el resultado: La segunda consulta muestra un nuevo registro y permite localizar el hash requerido.

Errores frecuentes: Tú mismo marcaste esta parte «mal redactado y hecho». Se ha conservado la secuencia de tu archivo, pero no debe tomarse como procedimiento validado sin repetirla en tu versión de WebGoat.



Captura del PDF original, pág. 19: Segunda consulta del endpoint y aparición del nuevo registro según la documentación.

Aclaración HTTP

Accept: application/json pide una respuesta JSON. Content-Type: application/json declara que el cuerpo enviado es JSON. Cambiar «el body a application/json» no es una descripción técnica precisa; son conceptos distintos.

08 · Spoofing an Authentication Cookie

En tu PDF: pp. 20-22

La cadena de transformaciones que debes recordar

En el ejercicio 2 probaste que la cookie de autenticación se podía reconstruir a partir de datos conocidos. Primero analizaste dos cookies, encontraste el patrón en el texto decodificado y luego fabricaste una cookie para el usuario tom.

```
Cookie original
    |  Decode as Base64
    v
Datos en hexadecimal
    |  Decode as ASCII hex
    v
Texto plano (patrón y nombre invertido)
    |  Modificar el texto para «tom»
    |  Encode as ASCII hex
    |  Encode as Base64
    v
Cookie nueva
```

Ejercicio 2: reconstruir cookie | PDF pp. 20-21

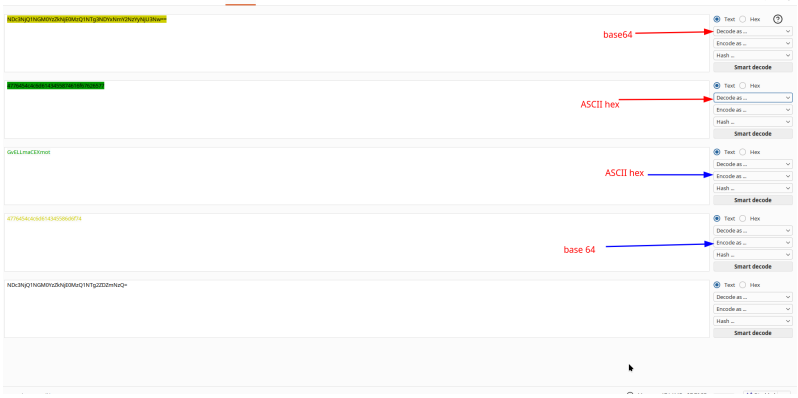
Objetivo: Imitar la autenticación de un usuario construyendo una cookie que la aplicación acepte.

Procedimiento de tu práctica

1. Captura la cookie de dos usuarios distintos en el laboratorio.
2. Cópialas al módulo Decoder de Burp.
3. Para cada una, aplica Decode as Base64 y después Decode as ASCII hex.
4. Compara los textos: en tus notas aparece una parte fija y el nombre escrito al revés.
5. Construye el nuevo texto usando «tom», manteniendo el patrón observado.
6. Aplica las operaciones inversas, primero Encode as ASCII hex y luego Encode as Base64.
7. Sustituye la cookie en la petición POST de prueba y envíala.

Por qué funciona: Las capas de codificación se pueden invertir sin conocer una clave secreta: no garantizan autenticidad de la cookie.

Cómo reconocer el resultado: La respuesta de WebGoat indica que la identidad forjada ha sido aceptada.



Captura del PDF original, pág. 21: Transformaciones de Decoder que usaste para construir una cookie de autenticación.

> **Diferencias esenciales**
>
> Codificación: transforma formato y se revierte (Base64, hex). Cifrado: pretende ocultar información usando una clave. Hash: produce una huella que no se «descifra». Una cookie debe protegerse además contra modificaciones, por ejemplo con una firma o un estado de sesión robusto.

09 · Crypto Basics: Base64, XOR y hashes

En tu PDF: pp. 23-26

Crypto Basics 2: Base64 | PDF pp. 23

Objetivo: Obtener usuario y contraseña ocultos en una cadena codificada.

Procedimiento de tu práctica

1. Copia la cadena ofrecida por WebGoat.
2. En Burp Decoder selecciona Decode as Base64.
3. Lee el resultado como texto para localizar usuario y contraseña.
4. Introduce los valores en el ejercicio.

Por qué funciona: Base64 es una codificación reversible, no un cifrado ni una protección para credenciales.

Cómo reconocer el resultado: El resultado decodificado contiene los datos pedidos.

Crypto Basics 3: WebSphere encoded password | PDF pp. 24

Objetivo: Entender y revertir una contraseña representada con la codificación utilizada en el ejemplo de WebSphere.

Procedimiento de tu práctica

1. Copia la cadena codificada que aparece en tu laboratorio.
2. Tus notas utilizan un decodificador de «WebSphere encoded password» para recuperarla.
3. Introduce la contraseña obtenida en WebGoat.

Por qué funciona: El ejercicio muestra que una representación reversible de contraseña no equivale a almacenarla de forma segura.

Cómo reconocer el resultado: La clave obtenida es aceptada por la actividad.

Aclaración técnica: en la explicación de clase también apareció {xor}. XOR puede ser reversible cuando se conoce la clave o el esquema; esto no significa que cualquier dato XOR deba ser trivialmente recuperable.

Crypto Basics 4: Plain Hashing y Salted Hashes | PDF pp. 25-26

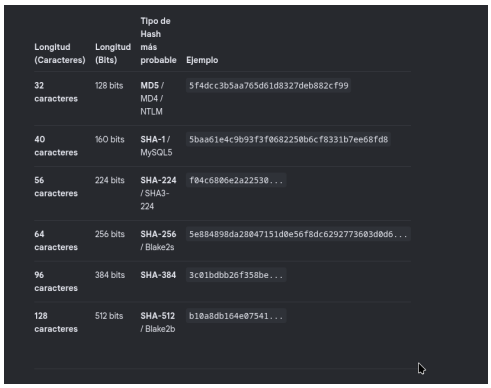
Objetivo: Identificar hashes y buscar una coincidencia conocida.

Procedimiento de tu práctica

1. Copia los hashes del enunciado.
2. Utiliza la longitud y los caracteres para reconocer algoritmos posibles.
3. En el documento comparaste el hash con bases de datos públicas de hashes conocidos.
4. Anotaste las palabras encontradas: secret para MD5 y password para SHA-256.

Por qué funciona: Los hashes no se decodifican directamente: puedes calcular el hash de una palabra candidata y compararlo, o reconocer un valor ya indexado.

Cómo reconocer el resultado: Los candidatos producen los hashes esperados y permiten completar el ejercicio.



Captura del PDF original, pág. 25: Tu tabla de longitudes aproximadas usada para identificar hashes por su formato.

Hexadecimal (caracteres)	Algoritmo probable
32	MD5 (128 bits)
40	SHA-1 (160 bits)
56	SHA-224 (224 bits)
64	SHA-256 (256 bits)
96	SHA-384 (384 bits)
128	SHA-512 (512 bits)

Importante para responder preguntas

La longitud solo orienta. Varios algoritmos pueden producir la misma longitud. Un hash con sal (salt) no se identifica correctamente por una tabla de contraseñas comunes si no se dispone del formato y los parámetros del sistema.

10 · SQL: fundamentos para los ejercicios intro

En tu PDF: pp. 32-38

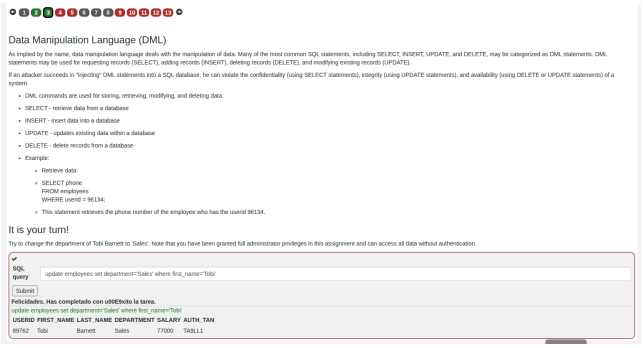
Qué es SQLi y qué necesitas comprender antes

SQL Injection aparece cuando la aplicación construye consultas SQL incorporando entradas no confiables como parte de la instrucción. Entonces el dato del usuario puede cambiar la lógica de la consulta. En tus prácticas se estudia primero sintaxis SQL y después cómo manipular las consultas de WebGoat.

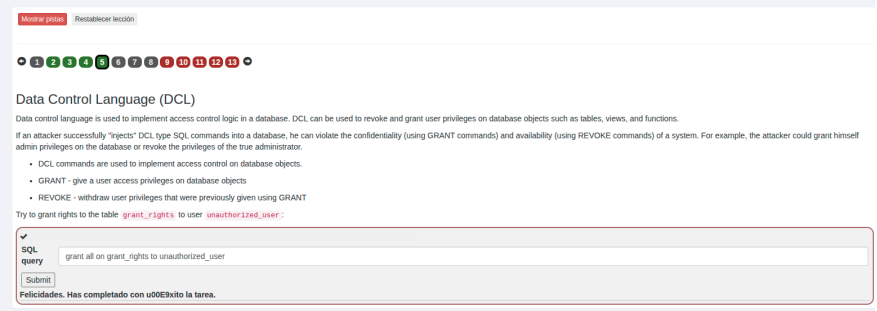
SQL Injection (intro), ejercicios 2-5

Ejercicio	Qué repasas	Ejemplo explicativo, no solución literal
2	SELECT / consultas	SELECT * FROM employees;
3	DML: INSERT, UPDATE, DELETE	UPDATE employees SET salary=... WHERE ...;
4	DDL: CREATE, ALTER, DROP	ALTER TABLE employees ADD ...;
5	DCL: GRANT y REVOKE	GRANT SELECT ON ... TO ...;

Nota de fidelidad: las páginas 32-34 muestran pantallas con instrucciones y SQL, pero tu documento no transcribe en texto todas las sentencias exactas. He separado los ejemplos de repaso de las respuestas concretas que solo aparecen en esas capturas.



Captura del PDF original, pág. 33: Captura del ejercicio SQL introductorio 3 sobre manipulación de datos.



Captura del PDF original, pág. 34: Captura del ejercicio SQL introductorio 5 sobre permisos y comandos DCL.

Para leer una consulta de ejemplo

```
SELECT first_name, last_name
FROM employees
WHERE last_name = 'Smith';
```

• SELECT: las columnas que queremos obtener.

• FROM: la tabla consultada.

• WHERE: condición para seleccionar registros.

• Una comilla simple: puede delimitar una cadena de texto, dependiendo del SQL.

SQL Injection (intro), ejercicio 9: String SQL Injection

El formulario espera una cadena de texto que se inserta en una consulta. La inyección puede cerrar esa cadena e introducir otra condición. Un ejemplo habitual de lógica siempre verdadera es:

```
' OR '1'='1
```

Si la aplicación incorpora ese texto sin parametrizar, puede modificar el resultado de una búsqueda. No todas las aplicaciones ni todos los campos son vulnerables; depende de cómo se construye la consulta.

Ejercicio 10: Numeric SQL Injection

Aquí el dato introducido se utiliza como un número, por lo que un ejemplo típico de comparación lógica sería:

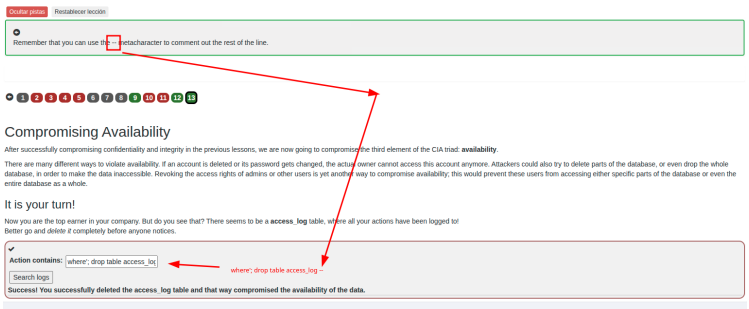
```
0 OR 1=1
```

La diferencia importante es que puede no requerirse cerrar comillas si el valor no está dentro de una cadena. Aun así, se debe observar el SQL concreto de la actividad.

Ejercicios 11, 12 y 13: CIA en una base de datos

Ejercicio	Qué se compromete	Señal típica
11	Confidencialidad	Acceso a datos que no debería devolver la consulta.
12	Integridad	Cambio de datos, por ejemplo mediante una actualización o una consulta de modificación.
13	Disponibilidad	Efecto sobre los datos o la funcionalidad que los usuarios necesitan.

Tu ejercicio 13 consulta primero un registro conocido y luego muestra una instrucción que produce un cambio de disponibilidad. Tus notas preguntan específicamente por --, que veremos a continuación.



Captura del PDF original, pág. 38: Captura posterior del ejercicio 13; la pantalla muestra el resultado de la segunda petición.

11 · SQL Injection avanzada: UNION SELECT

En tu PDF: pp. 39-43

La consulta exacta que aparece en tu documento

```
Smith' UNION SELECT userid, user_name, password,
'a', 'b', 'c', 1 FROM user_system_data --
```

Esta cadena está en tu página 40 y se explica extensamente en las páginas 41-43. Debes entender cada bloque, no aprenderla como si funcionara en cualquier web.

Fragmento	Función
Smith'	Cierra el valor de texto original (last_name = ...).
UNION SELECT	Agrega filas de una segunda consulta a la primera.
userid, user_name, password	Tres valores reales que interesan de user_system_data.
'a', 'b', 'c', 1	Cuatro constantes de relleno para llegar a siete columnas.
FROM user_system_data	Tabla adicional consultada.
--	Comenta lo que la aplicación añada después en la misma línea.

¿Por qué precisamente siete columnas?

En tu ejemplo `user_data` devuelve siete columnas y `user_system_data` tiene cuatro. Para usar UNION, ambas consultas deben devolver el mismo número de columnas, con tipos de datos compatibles por posición. La práctica necesita tres campos de verdad y completa los cuatro restantes con valores constantes.

```
SELECT * FROM user_data WHERE last_name = 'Smith'
UNION SELECT userid, user_name, password,
       'a', 'b', 'c', 1
FROM user_system_data -- '
```

El último signo `'` que añade la aplicación queda comentado. Así se evita que rompa la sintaxis de la consulta alterada.

> **Respuesta corta para el examen**
>
> -- no borra el WHERE ni comenta la inyección. Solo indica que el resto de la línea SQL es comentario. En este ejercicio evita que la comilla final añadida automáticamente provoque un error.

Errores frecuentes

- Poner seis u ocho columnas en el UNION cuando la consulta original devuelve siete.
- Utilizar un literal numérico en una columna que requiere otro tipo incompatible.
- Olvidar cerrar la comilla de `Smith` antes del UNION.

• Pensar que los valores `a, b, c, 1` son nombres de columnas; son constantes.



Captura del PDF original, pág. 40: La pantalla de «Pulling data from other tables» en la que probaste UNION SELECT.

12 · Blind SQL Injection: inferir información

En tu PDF: pp. 44-55

Qué significa «blind»

En Blind SQL Injection la aplicación no imprime la información obtenida directamente. Por eso la interrogas de forma indirecta y deduces la respuesta observando si cambia el texto, el estado, la longitud o el tiempo de respuesta.

```
4 AND 1=1     -> condición verdadera
4 AND 1=2     -> condición falsa
```

Esto solo es informativo si la aplicación responde de forma distinguible. Si hay variaciones visibles, se habla de blind basada en contenido o condiciones; si no, a veces se intenta deducir por el tiempo de procesamiento.

Técnica	Qué observas
Content/boolean-based	Una respuesta distinta cuando la condición es verdadera frente a falsa.
Time-based	Un retraso condicional que permite inferir si la condición se cumple.

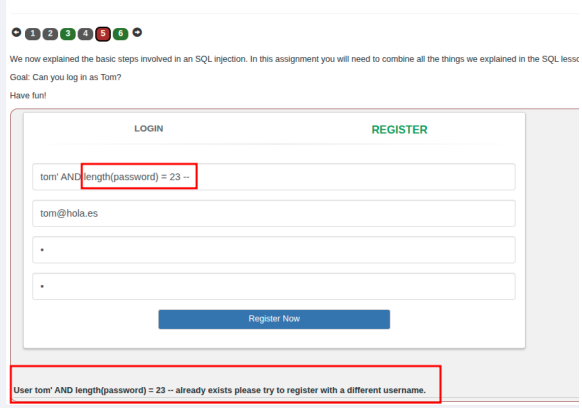
> **Muy importante**
>
> La frase «already exists» o «created» es el indicador que utiliza el ejercicio de registro de WebGoat. En otra aplicación podría ser al revés; primero debes calibrar qué respuesta corresponde a TRUE y cuál a FALSE.

Ejercicio 5, paso 1: averiguar la longitud de la contraseña

En las páginas 50-51 documentaste pruebas de longitud hasta encontrar un cambio de respuesta en 23. En el ejercicio concreto es la longitud que luego usaste para recorrer la contraseña carácter por carácter.

```
... AND length(password) = 23
```

Qué razonas: si la condición es verdadera, el formulario responde con el mensaje correspondiente a que ese nombre ya existe; si es falsa, aparece el mensaje alternativo. La consulta real de la práctica incluye contexto adicional y puede usar otra sintaxis; aquí se muestra únicamente la condición central.



Captura del PDF original, pág. 50: Comparación de las respuestas al probar la longitud de la contraseña.

Paso 2: encontrar el primer carácter

Usas `substring(password,1,1)` para extraer un solo carácter a partir de la posición 1. Lo comparas con distintas letras. En tus notas identificaste que el primer carácter era t.

```
... AND substring(password,1,1) = 't'
```

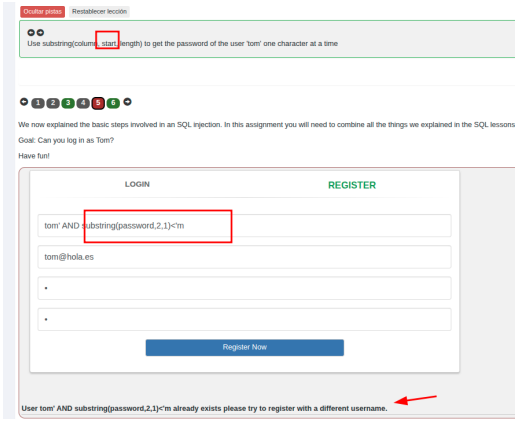
Esto se repite con `substring(password,2,1)` para el segundo carácter, luego la posición 3, etc. Cambiar el segundo argumento selecciona la posición; el tercer argumento indica cuántos caracteres extraes.

Paso 3: reducir intentos mediante comparaciones

En la página 52 utilizas < y > para comparar letras. Si la comparación de caracteres sigue la ordenación esperada, puedes descartar grupos y no probar todas las letras secuencialmente.

```
... AND substring(password,2,1) < 'm'
... AND substring(password,2,1) > 'f'
```

En lugar de aprender de memoria qué significa cada mensaje, vuelve a hacer una condición que sepas que es verdadera y otra falsa y compara con ambas.



Captura del PDF original, pág. 52: Tu comparación de la segunda posición con operadores de ordenación.

Paso 4: repetir las posiciones hasta completar la cadena

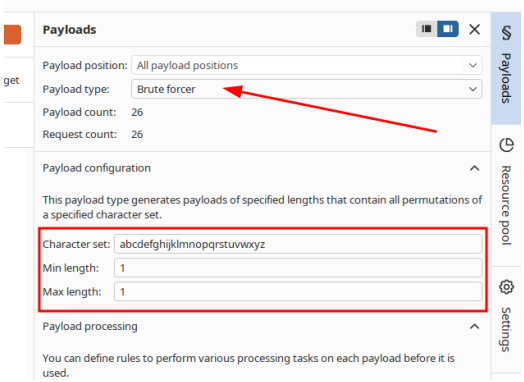
La contraseña que anotaste como resultado de esta versión del ejercicio es:

```
thisisasecretfortomonly
```

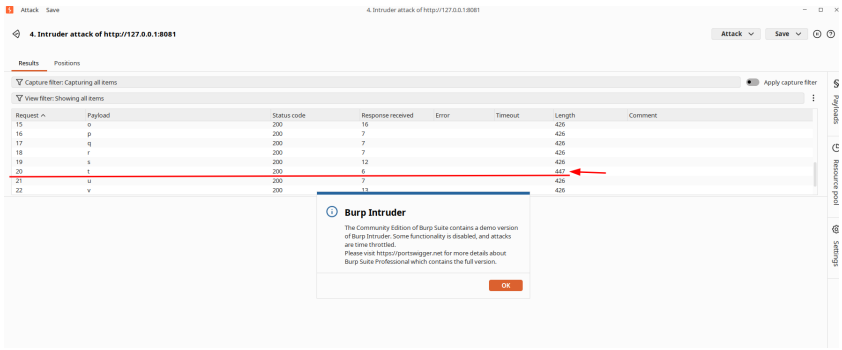
Tiene 23 caracteres. Es la respuesta de tu práctica, no una contraseña universal del reto: en otra instancia o versión puede variar.

Paso 5: automatizar letras con Burp Intruder

1. Captura la petición que contiene la condición de `substring`.
2. Envíala a Intruder y coloca la posición de payload alrededor de la letra que quieres probar.
3. En Payloads usa el método que anotaste como Brute force para probar caracteres individuales.
4. Configura un conjunto de caracteres apropiado para el ejercicio y longitudes mínima y máxima de 1.
5. Inicia la prueba. Ordena por Length y compara la respuesta distinta con el patrón TRUE/FALSE que mediste.
6. Para el carácter siguiente cambia la posición de substring (1, 2, 3...) y repite.



Captura del PDF original, pág. 55: Configuración «Brute force» de un carácter en Burp Intruder, según tu ejercicio.



Captura del PDF original, pág. 55: Análisis de la respuesta diferente en Intruder; la longitud es una pista que se comprueba leyendo el contenido.

Lo que pueden preguntar

¿Por qué una letra concreta aparece con un Length distinto? Porque esa petición hizo que el servidor produjera una respuesta diferente; debes leer el texto para decidir si se corresponde con la condición verdadera.

13 · XSS: Reflected, DOM y Stored

En tu PDF: pp. 57-65

Qué ocurre cuando existe XSS

Cross-Site Scripting permite que información controlada por un atacante sea interpretada como HTML o JavaScript en el navegador. En WebGoat se utiliza una alerta o una función de comprobación para demostrar ejecución. El riesgo real depende de qué puede hacer el código dentro del contexto de la página.

Tipo	Cómo reconocerlo
Reflected	El dato entra en una petición y vuelve en la respuesta de esa misma operación.
DOM-based	Un JavaScript del cliente lee la entrada y la introduce en el DOM de manera insegura.
Stored	El payload queda guardado y puede ejecutarse cuando la página se vuelve a consultar.
Self-XSS	El propio usuario ejecuta directamente el código en su sesión, sin que eso pruebe que pueda afectar a otros.

Reflected XSS, ejercicio 7: salir del contexto HTML

En tu página 60 aparece este payload de prueba:

```
"><script>alert(1)</script>
```

El prefijo "> busca cerrar un atributo o etiqueta previa en determinados contextos. Después aparece una etiqueta de script que llama a `alert(1)`. Si el navegador muestra la alerta por la respuesta de la aplicación, se ha demostrado ejecución de JavaScript en ese contexto.

Reflected XSS, ejercicio 7 | PDF pp. 59-60

Objetivo: Conseguir que WebGoat refleje y ejecute un script sencillo.

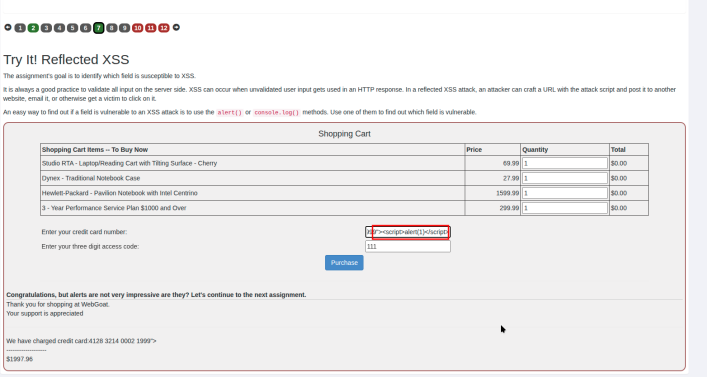
Procedimiento de tu práctica

1. Introduce el payload en el campo vulnerable que aparece en el formulario.
2. Envía el formulario.
3. Comprueba si el contenido se refleja sin codificar en el HTML.
4. Verifica que el navegador ejecuta `alert(1)`.

Por qué funciona: El servidor no codifica correctamente la entrada antes de situarla en el HTML.

Cómo reconocer el resultado: Aparece la alerta y WebGoat acepta el intento.

Errores frecuentes: El prefijo de cierre depende del contexto HTML del campo: no es necesario para todos los XSS.



Captura del PDF original, pág. 60: Uso del payload de Reflected XSS en el formulario de WebGoat.

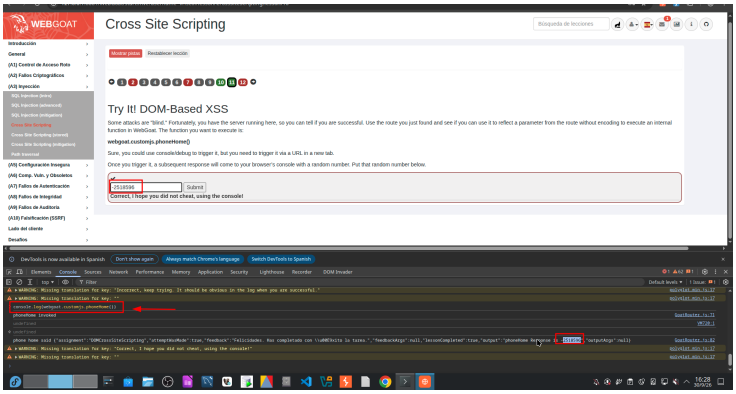
DOM XSS, ejercicios 10 y 11: encontrar ruta y función

En tus capturas de la página 63 abres Developer Tools y buscas código que trata datos de una ruta. El objetivo que documentaste en otras notas de WebGoat es ejecutar la función interna:

```
webgoat.customjs.phoneHome()
```

1. Inspecciona los archivos JS y el comportamiento de la ruta del ejercicio con DevTools.
2. Determina qué parte de la URL puede controlarse y dónde se introduce en el DOM.
3. Construye una URL de laboratorio que active la llamada a `phoneHome()` a través de la vulnerabilidad; no vale ejecutarla manualmente desde la consola.
4. Abre esa URL en una pestaña nueva.
5. Revisa la consola: WebGoat devuelve un número de confirmación. Escríbelo en la actividad.

Límite de las notas: el texto de tu PDF no transcribe la ruta completa ni el payload final del ejercicio DOM. Las capturas muestran los pasos, pero no sería responsable inventar aquí una URL exacta que no está documentada en texto.



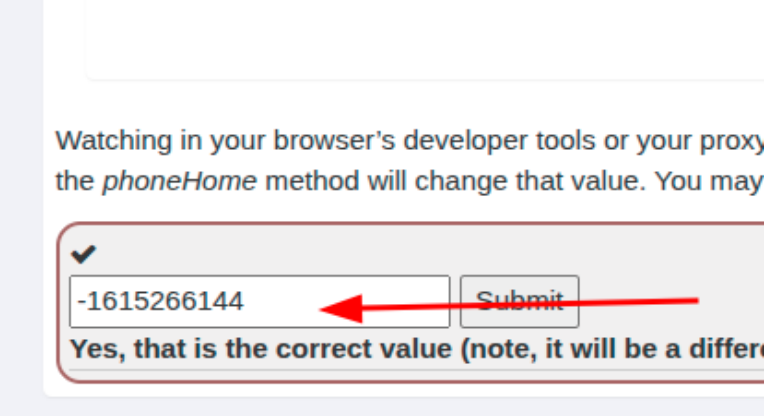
Captura del PDF original, pág. 63: DevTools durante la búsqueda de la ruta vulnerable para la prueba DOM-Based XSS.

Stored XSS, ejercicio 3: código almacenado

En tus páginas 64-65 aparece la práctica con la comprobación de phoneHome y un código aleatorio. A diferencia del Reflected XSS, el payload se guarda en la aplicación y se ejecuta al mostrarlo posteriormente.

1. Localiza el campo que WebGoat almacena y enseña después.
2. Introduce el contenido activo de prueba según el contexto del ejercicio.
3. Guarda el dato y vuelve a la vista que lo muestra.

4. Comprueba la ejecución mediante la consola o el proxy.
5. Introduce el número de confirmación, que puede variar entre intentos.



Captura del PDF original, pág. 65: WebGoat confirma el valor devuelto por phoneHome en tu ejercicio de XSS almacenado.

> **Cómo distinguirlos en 10 segundos**
>
> Reflected: respuesta inmediata. Stored: persiste en comentarios/datos. DOM: la ejecución depende del JavaScript que modifica la página. DOM y Stored/Reflected no siempre son categorías completamente excluyentes.

14 · Path Traversal: cómo funcionan las rutas

En tu PDF: pp. 70-75

La idea principal: qué hace ../

Un Path Traversal ocurre cuando el servidor acepta fragmentos de ruta controlados por el usuario y permite escapar del directorio autorizado. Cada segmento `../` sube un nivel. El directorio de partida es el que usa el servidor para resolver el archivo, no la carpeta local desde la que pruebas.

```
/home/webgoat/uploads/imagen.jpg
/home/webgoat/uploads/../secreto.txt
                  |
                  +-- sube al directorio «webgoat»
```

Ejercicios 2 y 3: Path traversal while uploading files

Tus capturas muestran que la aplicación utiliza un campo Full Name relacionado con el nombre o la ubicación final de una imagen de perfil. Introdujiste una ruta con segmentos de navegación para intentar modificar el directorio de escritura.

1. Abre la actividad de subida de imagen.
2. Observa qué nombre o ruta utiliza WebGoat por defecto.
3. Prueba el campo permitido en el laboratorio con una ruta relativa que incluya `../`.
4. Sube el archivo y lee el resultado: el servidor puede mostrar el lugar de almacenamiento.
5. Compara la ruta prevista con la ruta obtenida para entender si hubo traversal.

Ejercicio 4: manipular la petición multipart

En la página 72 interceptaste la subida con Burp. Esto ayuda cuando el navegador muestra un formulario pero el servidor recibe un multipart/form-data con nombre del archivo, campos y límites de cada parte.

1. Sube una imagen normal para provocar la petición.
2. Abre el POST multipart en Burp y localiza la parte del archivo/metadata.
3. Modifica únicamente el campo relacionado con la ruta o el nombre que comprueba el ejercicio.
4. Envía con Repeater y examina la respuesta de WebGoat.

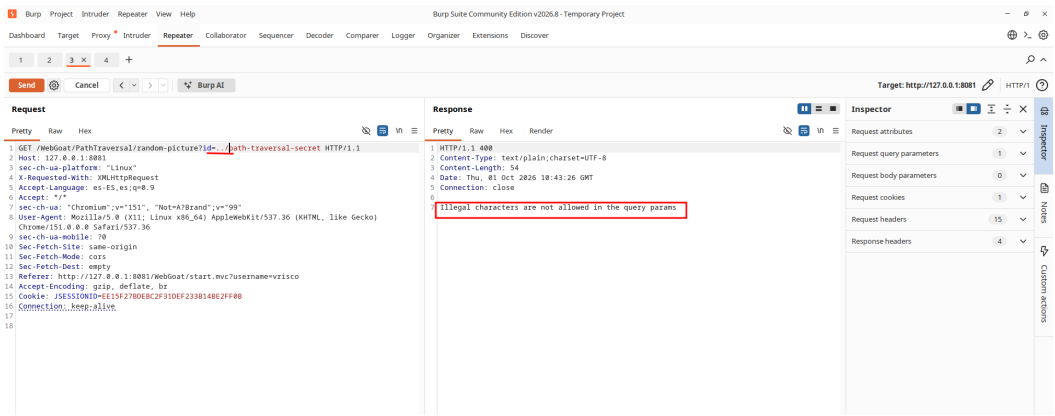
Recuerda distinguir el parámetro `filename=` de cada parte multipart y cualquier campo adicional de la aplicación. Una modificación en un sitio distinto puede no afectar a la ruta que usa el servidor.

Ejercicio 5: URL Encoding para sortear el filtro

```
../             ->   %2e%2e%2f
../../          ->   %2e%2e%2f%2e%2e%2f
..%2f           ->   ../  (tras decodificación URL)
```

Tu actividad usa una petición a random-picture. El servidor bloquea la representación literal `../`, pero en el ejemplo de clase se prueba la secuencia codificada `%2e%2e%2f`.

1. Captura la solicitud relacionada con la imagen aleatoria.
2. Identifica qué fragmento va después de `random-picture`.
3. Consulta la respuesta para descubrir la ubicación que te indica WebGoat.
4. En el punto de entrada vulnerable, reemplaza los segmentos `../` por su forma URL encoded si el filtro literal los bloquea.
5. No dupliques el sufijo `.jpg` cuando la propia ruta del ejercicio ya lo añade.
6. Compara la respuesta con la esperada: en tu documentación se obtiene un valor asociado al nombre del usuario, representado en SHA-512.



Captura del PDF original, pág. 74: Burp muestra la respuesta al intentar modificar el parámetro de random-picture.

Por qué importa la codificación

Un filtro que compara texto antes de decodificar puede no reconocer una ruta que luego el servidor interpretará como `../`. Una validación segura se realiza sobre la ruta normalizada y con límites del directorio permitidos.

15 · Zip Slip: Path Traversal dentro de un ZIP

En tu PDF: pp. 76-78

Qué lo diferencia de subir una imagen normal

En Zip Slip el atacante no necesita que el campo visible contenga `../`; el recorrido está en el nombre de una entrada dentro del ZIP. El programa extractor vulnerable utiliza ese nombre para escribir fuera de su carpeta de extracción.

Ejercicio 7: tu caso, paso a paso

1. Descarga una imagen y crea un ZIP normal para comprobar cómo WebGoat extrae sus archivos.
2. Observa la ruta de destino que aparece después de la extracción.
3. Construye otro ZIP cuyo nombre de entrada incluya varios segmentos `../` seguidos de la ruta objetivo.
4. En tu práctica el archivo final apuntaba a `/home/webgoat/.webgoat-2026.3.1/PathTraversal/vrisco/vrisco.jpg`.
5. Sube el ZIP al ejercicio y verifica si sobrescribe la imagen correspondiente.

```
../../../../../../../../home/webgoat/
.webgoat-2026.3.1/PathTraversal/vrisco/vrisco.jpg
```

¿Por qué hay ocho ../ si con cinco bastaba?

Tus notas sitúan la extracción en `/home/webgoat/.webgoat-2026.3.1/PathTraversal/vrisco/`. Para salir desde ahí hasta la raíz se sube:

```
1  ../             -> .../PathTraversal/
2  ../../          -> .../.webgoat-2026.3.1/
3  ../../../       -> /home/webgoat/
4  ../../../../    -> /home/
5  ../../../../../ -> /
```

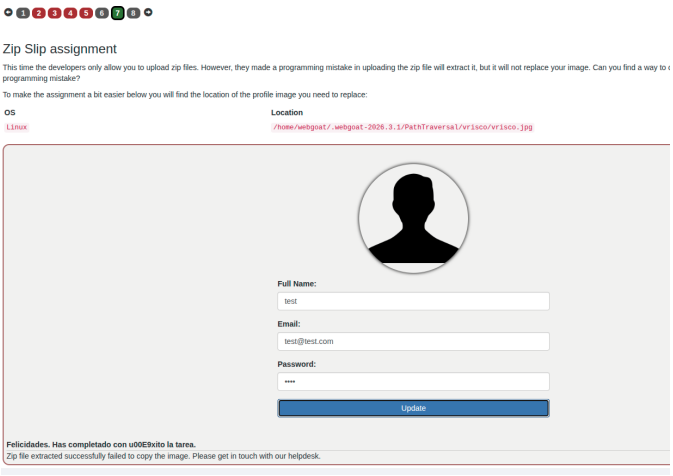
Cinco serían suficientes en ese punto de partida. Los tres adicionales se pusieron como margen por si la extracción real comenzara más abajo. En una ruta Linux normal no se puede subir por encima de `/`. Un extractor seguro bloquearía una entrada que salga del directorio autorizado.

Ejemplo complementario: crear un ZIP con nombre de entrada especial

Tu PDF muestra el uso de comandos para preparar el ZIP. Esta variante en Python sirve para comprender qué parte es importante: el arcname guardado dentro del archivo comprimido. Úsala solamente en el laboratorio propio de WebGoat.

```
import zipfile
with zipfile.ZipFile('prueba.zip', 'w') as z:
    z.write('foto.jpg',
      '../../../../../../../../home/webgoat/'
      '.webgoat-2026.3.1/PathTraversal/vrisco/foto.jpg')
```

Por qué sirve: el primer argumento es el archivo local que metes en el ZIP; el segundo es el nombre que quedará guardado dentro. Ese segundo nombre es lo que debe validar un extractor.



Captura del PDF original, pág. 77: Pantalla del reto Zip Slip tras subir el archivo preparado, según tu documentación.

> **No mezcles dos ubicaciones**
>
> /home/victorrisco/prova es la carpeta donde preparaste el archivo. /home/webgoat/... es una ruta del servidor que lo extrae. Los `../` se cuentan desde el directorio remoto de extracción.

16 · CSRF: peticiones con la sesión de otro

En tu PDF: pp. 79-85

Entender la vulnerabilidad sin confundirse con XSS

CSRF (Cross-Site Request Forgery) hace que el navegador de un usuario ya autenticado envíe una petición que no quería realizar. El servidor recibe la cookie legítima, pero eso no demuestra que la acción haya sido deseada. No hace falta que el atacante conozca la contraseña de la víctima.

Ejercicio 3: Basic GET CSRF

Basic GET CSRF, ejercicio 3 | PDF pp. 79-80

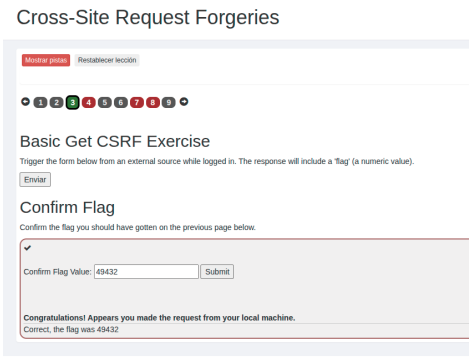
Objetivo: Activar una petición que el ejercicio espera desde una fuente externa y recuperar el flag.

Procedimiento de tu práctica

1. Deja la sesión de WebGoat iniciada.
2. Activa y captura la petición del reto de CSRF.
3. En Burp, tus notas indican quitar la cabecera Referer y reenviar la solicitud.
4. Recupera el flag numérico que devuelve el servidor.
5. Introduce ese flag en el campo Confirm Flag.

Por qué funciona: La práctica demuestra que una acción puede aceptar una petición que no procede de su flujo visual habitual.

Cómo reconocer el resultado: WebGoat acepta el valor generado y da por completada la actividad.



Captura del PDF original, pág. 80: Pantalla «Confirm Flag» de Basic GET CSRF en tu documento.

> **Aclaración muy importante**
>
> Eliminar Referer en Repeater es la maniobra que documentaste para este reto. No crea realmente una página externa ni es una prueba general de CSRF. Las protecciones serias no dependen exclusivamente de esa cabecera.

Ejercicio 4: petición como si viniera de otro contexto

Tu anotación dice que la solicitud debe realizarse «como si fueras otra persona» y vuelves a borrar Referer. Conservar el valor correcto de la cookie y las demás cabeceras es esencial para interpretar el comportamiento del laboratorio.

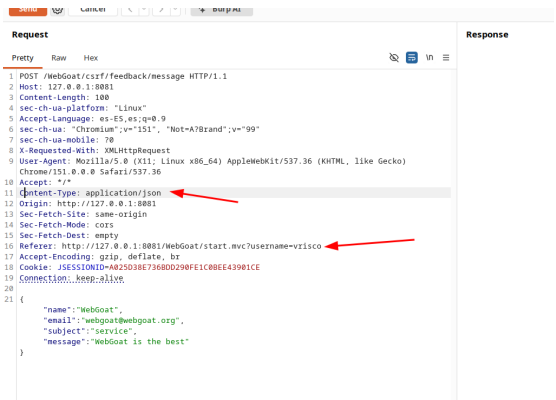
1. Reproduce la acción en el reto 4 y captura la petición.
2. Envíala a Repeater; comprueba método, cuerpo y cookie.
3. Prueba la modificación de Referer de tus notas.
4. Analiza la respuesta y verifica la aceptación en WebGoat.

Ejercicio 7: cambio de Content-Type

El PDF anota enviar la petición, quitar Referer y cambiar Content-Type por text/plain. Esto es relevante porque el servidor puede aceptar un tipo de contenido con menos restricciones de las esperadas.

```
Content-Type: text/plain
```

El contenido del body no se transforma mágicamente a texto plano por cambiar una cabecera. Lo importante es comprobar cómo lo procesa el servidor después de modificar esa declaración.



Captura del PDF original, pág. 82: La petición de la práctica con Content-Type y Referer resaltados.

Ejercicio 8: Login CSRF, dos pestañas y dos sesiones

Esta actividad está muy bien explicada en tu página 85. La secuencia es:

1. Abre la página del ejercicio 8 con la sesión de vrisco. No pulses el botón todavía.
2. Abre otra pestaña de WebGoat.
3. Crea e inicia sesión con el usuario csrf-vrisco.
4. Vuelve a la pestaña antigua sin recargarla.
5. Pulsa el botón de confirmación.
6. Comprueba que el servidor procesa la petición usando la cookie actual de csrf-vrisco, aunque la pestaña se abrió cuando estabas como vrisco.

> **Idea del ejercicio**
>
> Las pestañas del mismo navegador comparten las cookies de ese sitio. El HTML antiguo no conserva una cookie anterior: al hacer una nueva petición el navegador utiliza el estado de sesión vigente.

17 · Security Misconfiguration

En tu PDF: pp. 86-87

Qué significa configuración insegura

Una aplicación puede tener código correcto y aun así revelar información por una configuración inapropiada: credenciales predefinidas, errores detallados, debug activado, servicios de prueba disponibles o endpoints internos expuestos.

Ejercicio 2: default credentials | PDF pp. 86

Objetivo: Comprobar que existen credenciales por defecto que no se cambiaron.

Procedimiento de tu práctica

1. Abre el formulario del ejercicio.
2. Prueba las credenciales habituales sugeridas por el propio reto.
3. Observa si se puede iniciar sesión en el servicio de demostración.

Por qué funciona: Las cuentas que llegan con un producto o entorno pueden seguir habilitadas si no se cambian.

Cómo reconocer el resultado: El sistema de prueba acepta credenciales que no deberían estar activas en producción.

Ejercicio 3: Verbose Debug Mode Leakage | PDF pp. 86

Objetivo: Provocar un error que revele información interna.

Procedimiento de tu práctica

1. Pulsa el botón que fuerza la excepción.
2. Lee la salida detallada que devuelve la aplicación.
3. Localiza los datos sensibles o el token que el enunciado pide.

4. Completa la comprobación utilizando ese valor dentro del reto.

Por qué funciona: El modo debug o la respuesta de error muestran información interna que solo debería guardarse en logs privados.

Cómo reconocer el resultado: El contenido del error contiene una pista suficiente para completar la actividad.

Ejercicio 4: Exposed Actuator Endpoints | PDF pp. 87

Objetivo: Descubrir información expuesta por funciones internas de administración o diagnóstico.

Procedimiento de tu práctica

1. Pulsa la acción indicada para provocar la respuesta del endpoint.

2. Examina los datos devueltos y localiza los valores que el reto señala.

3. Distingue entre información de diagnóstico útil y datos sensibles que no deberían ser públicos.

Por qué funciona: Las rutas de diagnóstico expuestas pueden revelar el estado o la configuración de la aplicación.

Cómo reconocer el resultado: La respuesta contiene información que WebGoat considera una filtración.

Ejercicio 5: Hardening de configuración | PDF pp. 87

Objetivo: Reconocer ajustes que deberían aplicarse para reducir filtraciones.

Procedimiento de tu práctica

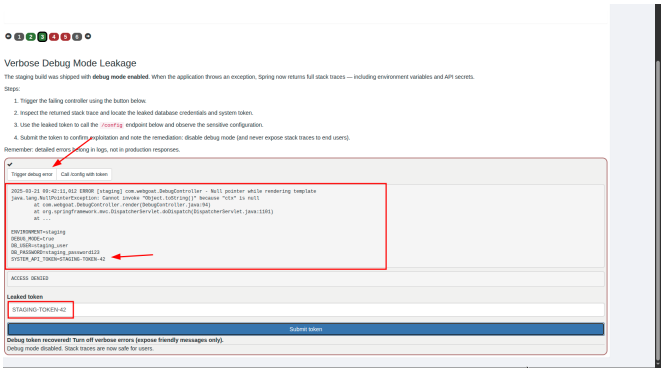
1. Revisa las opciones de endurecimiento que muestra la actividad.

2. Identifica configuraciones relacionadas con debug, errores, credenciales, secretos y rutas internas.

3. Relaciona cada control con el riesgo que evita.

Por qué funciona: Una configuración segura reduce la información expuesta incluso si se produce una excepción.

Cómo reconocer el resultado: La actividad permite reconocer qué opciones endurecen el despliegue.



Captura del PDF original, pág. 86: Captura del ejercicio de errores verbosos que devuelve información interna.

> **Frase que debes saber**
>
> Los errores detallados deben registrarse en los logs protegidos del servidor; al usuario se le debe mostrar un error genérico, sin secretos ni trazas internas.

18 · Tabla de decisión y checklist de Burp

Identifica la vulnerabilidad a partir de la pista

Lo que observas	Qué tema repasar
Una cookie válida permite utilizar una sesión ajena	Hijack a Session
Cambias el ID y ves datos de otro perfil	IDOR
Una función no aparece en menú, pero la ruta responde	Missing Function Level Access Control
Cookie = Base64 y hex reversible	Spoofing an Authentication Cookie
Se muestra una segunda tabla con UNION SELECT	SQL Injection avanzada
No ves datos, pero hay respuestas TRUE/FALSE	Blind SQL Injection
Tu entrada ejecuta un script en la respuesta	Reflected XSS
Una entrada persistente ejecuta script en futuras visitas	Stored XSS
JS de la página interpreta un fragmento de URL en el DOM	DOM-Based XSS
Se sale de una ruta con ../ o %2e%2e%2f	Path Traversal
Dentro del ZIP aparece una ruta con ../	Zip Slip
Se usa una sesión válida para una acción involuntaria	CSRF
El error devuelve secretos o stack traces	Security Misconfiguration

Checklist antes de dar por resuelto un ejercicio

☐ He identificado la petición correcta y he guardado el original para comparar.

☐ Sé qué entrada controlo (URL, body, cookie, cabecera, ruta o contenido ZIP).

☐ He cambiado solo la parte que afecta al ejercicio.

☐ He entendido qué resultado significa éxito, no solo un cambio visual.

☐ He revisado el contenido de la respuesta además de Status y Length.

☐ Puedo explicar con una frase qué comprobación de seguridad faltó en la aplicación.

19 · Hoja de memoria: comandos, payloads y diferencias

Secuencias que conviene reconocer

```
Burp:      Proxy → Repeater → Intruder
HTTP:      GET /ruta?param=valor
           Content-Type: application/json
Cookie:    Decode Base64 → Decode ASCII hex
           Encode ASCII hex → Encode Base64
SQL:       OR 1=1    /    AND 1=2    /    --
UNION:     UNION SELECT columna1, columna2, ...
Blind:     LENGTH(...) / SUBSTRING(...,pos,1)
XSS:       <script>alert(1)</script>
WebGoat:   webgoat.customjs.phoneHome()
Path:      ../  =  %2e%2e%2f
Zip Slip:  ZIP con nombres de entrada ../
```

Cuatro diferencias que suelen caer

Comparación	Respuesta correcta
GET vs POST	Método HTTP y ubicación habitual de parámetros; ninguno evita por sí solo una vulnerabilidad.
Reflected vs Stored XSS	En Reflected la entrada vuelve en esa respuesta; en Stored se guarda y reaparece.
IDOR vs control de función	IDOR afecta el objeto consultado; la función oculta afecta al endpoint/acción no autorizada.
Encoding vs Hashing	Encoding se revierte; un hash se compara contra candidatos, no se decodifica.
Path Traversal vs Zip Slip	Path Traversal utiliza rutas con ../; Zip Slip las contiene en entradas de un ZIP.

Errores que conviene corregir al estudiar

- Length distinto ≠ éxito confirmado: siempre hay que abrir la respuesta.
- -- no ignora todo SQL: comenta lo que viene después, no lo anterior.
- Referer ausente ≠ sitio externo demostrado: el laboratorio solo valida su lógica concreta.
- Base64 ≠ cifrado: es una representación reversible.
- Más ../ no significa más permisos: solo cambia cómo se resuelve una ruta.
- MD5 y SHA no se descifran: buscar una coincidencia no revierte el algoritmo.

20 · Simulacro de examen: 20 preguntas

01. Ves un POST en Burp y WebGoat te pide un GET. ¿Qué partes de la petición revisarías?

02. ¿Qué representa la línea en blanco en una solicitud HTTP/1.x?

03. ¿Para qué sirve el módulo Repeater y cuándo elegirías Intruder?

04. ¿Qué dos cosas comprobarías antes de concluir que un Length distinto es la solución?

05. Define confidencialidad, integridad y disponibilidad con un ejemplo de ataque a cada una.

06. En una cookie de sesión, ¿qué riesgo demuestra que el identificador sea predecible?

07. ¿Por qué cambiar un ID y obtener datos ajenos es IDOR?

08. ¿En qué se diferencia un endpoint administrativo oculto de un IDOR?

09. ¿Qué orden de transformaciones usaste para decodificar la cookie del ejercicio 2?

10. ¿Por qué una contraseña codificada con Base64 no está protegida?

11. ¿Por qué no puedes «descifrar» directamente un hash SHA-256?

12. ¿Por qué se escribieron las constantes a, b, c, 1 en el UNION SELECT?

13. ¿Qué hace -- al final del payload de la SQL Injection avanzada?

14. ¿Cómo calibrarías TRUE y FALSE antes de una Blind SQLi?

15. ¿Qué argumento cambia en SUBSTRING para probar la siguiente letra?

16. ¿Qué diferencia hay entre Reflected, DOM-Based y Stored XSS?

17. ¿Para qué utilizaste phoneHome() en WebGoat?

18. ¿Cómo representarías ../ mediante URL encoding?

19. ¿Desde qué directorio se cuentan los ../ en Zip Slip?

20. En Login CSRF, ¿por qué la pestaña antigua utiliza la sesión más reciente?

21 · Respuestas del simulacro

01. Método, URL, query string, cuerpo, cabeceras como Content-Type y los parámetros que deben moverse.

02. Separa el bloque de cabeceras del cuerpo del mensaje.

03. Repeater permite pruebas individuales y comparación; Intruder automatiza un rango de valores.

04. Abrir el cuerpo y verificar la función/resultado buscado; descartar errores o respuestas variables.

05. Confidencialidad = lectura autorizada (fuga de contraseñas); integridad = datos correctos (cambio no autorizado); disponibilidad = servicio accesible (DoS).

06. Que un tercero podría probar valores candidatos y apropiarse de una sesión válida.

07. El servidor entrega un objeto que pertenece a otro usuario sin comprobar autorización.

08. El segundo caso permite invocar funciones restringidas aunque el menú no las muestre; IDOR se refiere a objetos.

09. Base64 decode, después ASCII hex decode; para volver a codificar se aplica el orden inverso.

10. Cualquiera puede decodificarla; es una representación reversible, sin garantía de secreto.

11. El hash es una huella unidireccional: se prueban candidatos y se comparan los resultados.

12. Para alcanzar las siete columnas que exige la primera SELECT del UNION.

13. Comenta lo que el servidor añada después en esa línea, incluida la posible comilla final.

14. Probar una condición necesariamente verdadera (1=1) y otra falsa (1=2), y observar qué respuesta corresponde a cada una.

15. El argumento que indica la posición de inicio: substring(campo,posición,1).

16. Reflected: se refleja en una respuesta; DOM: JS lo inserta en el DOM; Stored: se guarda y se muestra luego.

17. Para demostrar ejecución JavaScript a través de la vulnerabilidad y obtener un número de comprobación.

18. %2e%2e%2f.

19. Desde la ruta donde el servidor extrae el ZIP; no desde la carpeta local.

20. Porque las pestañas del mismo sitio comparten cookies y cada petición utiliza la cookie vigente.

Apéndice · Qué falta o no está confirmado en el original

Esta guía no intenta convertir tus anotaciones en certezas que el documento no acredita. Estos son los puntos que conviene verificar en tu instalación si tienes tiempo antes del examen:

Parte	Estado en el original
HTTP Basic 3	Se ve el enunciado en captura, pero no están transcritas todas las condiciones exactas de la petición.
SQL intro 2-5	Las sentencias resueltas están en capturas pequeñas; los ejemplos añadidos aquí son de sintaxis general.
Missing Function Level Access Control 4	Tu documento indica expresamente que el procedimiento quedó mal redactado y hecho.
DOM XSS 10-11	No está transcrita la URL exacta ni el payload final de la ruta DOM.
CSRF 4 y 7	El texto anota cambios de cabeceras, pero no documenta todas las condiciones del ejercicio.
Examen	Los «NO SALE» son notas tuyas; no hay una confirmación del profesor dentro del PDF.

Fuente principal: «WebGoat Exercicis Victor Risco.pdf» (98 páginas, documento aportado). Capturas reproducidas de ese mismo material. Los ejemplos genéricos añadidos están indicados como tales o acompañados de notas aclaratorias.